

FitPick Stylist Capabilities Handoff

Date: July 11, 2026

Product: FitPick launch build

Primary route: /stylist

Primary API: POST /api/stylist/chat

Summary

FitPick Stylist is an authenticated wardrobe-grounded styling assistant. It answers style questions using the user's saved wardrobe, style profile, fashion memory, and deterministic outfit recommendation logic. It can suggest outfits from owned items, explain why a look works, help improve or adjust a look, identify wardrobe gaps, support packing/travel prompts, and optionally generate avatar or premium visual previews when the request produces a valid owned outfit.

The stylist is designed to be useful without inventing clothes the user does not own. Shopping advice is disabled by default and only appears when explicitly allowed by the API request.

What The Stylist Can Do Now

1. Answer wardrobe-grounded style questions

The stylist can answer prompts such as:

- What should I wear today?
- Style me for work in cold weather.
- Find a polished dinner look.
- Dress me for a warm workday.
- What should I wear to a wedding?
- Make this look more formal.
- Make this look more casual.
- Try another look from my wardrobe.

It uses:

- saved wardrobe items
- verified item metadata when available
- style profile / Style DNA
- fashion memory summary

- recent current-session chat messages
- deterministic outfit recommendation output

2. Recommend outfits from owned wardrobe items

For outfit-style prompts, the stylist builds a deterministic recommendation first, then asks the language model to explain it in a concise, fashion-aware way.

The response can include:

- recommended wardrobe item IDs
- outfit title
- occasion detected
- summary
- why the look works
- styling tips
- missing wardrobe categories
- completeness warnings
- fit/footwear warnings when relevant

The UI displays selected wardrobe items in a horizontal strip with names, categories, colors, and thumbnails when available.

3. Support shoe and accessory uploads

Users can upload and save shoes, bags, and accessories as first-class wardrobe items.

Supported wardrobe categories include:

- tops
- bottoms
- dresses
- native wear
- outerwear
- shoes
- bags
- accessories

Shoes and accessories should always remain part of the outfit data, wardrobe strip, saved look, and preview request when selected by the stylist. They must not be silently dropped from recommendations or visual preview generation.

4. Preserve wardrobe grounding

The stylist validates AI output against owned item IDs. If the AI returns item IDs that are not owned by the user, they are removed before the response is returned.

Current grounding rules:

- Do not invent item IDs.
- Do not invent garments, brands, colors, or sizes.
- Unknown values stay unknown.
- Full outfit recommendations must consider footwear.
- If shoes are missing, the stylist must not call the outfit complete.
- Shopping suggestions are excluded unless `allowShoppingAdvice` is true.

5. Use seasonal and global occasion context

The stylist prompt includes explicit support for:

- seasonal dressing
- winter layering
- rain and transitional weather
- warm weather
- work and commute
- school
- dinner and date night
- weddings and formal events
- travel and packing
- church
- cultural and traditional events when the user's wardrobe or request supports them

It should not force culturally specific styling unless the user's wardrobe or request supports it.

6. Support multiple intent types

The stylist classifies prompts into these intent types:

- `outfit_request`
- `compare_outfits`
- `improve_outfit`

- explain_item
- packing_help
- wardrobe_gap
- general_style_advice
- shopping_advice_requested
- unclear

Visualization is only considered for outfit-style intents with an owned outfit.

7. Generate avatar outfit previews

When enabled and appropriate, the stylist can persist the outfit recommendation and trigger a digital human/avatar preview.

Shoes, bags, and accessories are always included in the outfit payload passed to preview generation when they are part of the suggested outfit. If the visual provider cannot render a shoe or accessory accurately, FitPick should show a clear preview limitation instead of removing the item.

Target preview behavior:

- The avatar preview should use the user's selected avatar base: male, female, or not specified.
- The preview provider should receive the actual uploaded wardrobe item images for every selected outfit item.
- The provider should render the avatar wearing the suggested clothes, not a prompt-only approximation of similar clothes.
- Shoes, bags, and accessories should be passed as image-grounded outfit inputs whenever they are part of the selected outfit.
- If a provider cannot accurately place an item, FitPick should keep the item in the outfit and return a preview warning.

Current implementation note:

- The avatar preview path now attempts an image-grounded generation path when selected wardrobe item images are available and the configured image model supports image edits.
- In that path, actual wardrobe item images are attached as image inputs with high input fidelity where supported.
- If garment images are unavailable or the provider cannot accept image inputs, FitPick falls back to the prompt-based preview path and returns a preview warning.
- For even stronger exact-clothing fidelity, FitPick should still support a dedicated virtual try-on provider that accepts an avatar/person image plus garment image inputs.

Recommended provider contract:

```
generateTryOnPreview({
  avatarImageUrl,
  avatarBase,
  garmentImages,
  outfitItemIds,
  garmentCategories,
  posePreset
})
```

garmentImages should include tops, bottoms, dresses, native wear, outerwear, shoes, bags, and accessories when present in the outfit.

Default visual mode:

- digital_human

Other supported visual modes:

- premium_preview
- none

The preview system can return:

- not_started
- queued
- generating
- ready
- failed

When background jobs are enabled, preview generation is queued and processed by the worker. When jobs are disabled, the system attempts inline generation.

8. Reuse cached previews

Avatar and premium previews use cache keys based on:

- user ID
- outfit ID
- selected wardrobe item IDs
- avatar profile
- avatar base: male, female, or not specified
- visualization style
- pose preset

- prompt version

If a matching generated preview already exists, the stylist returns it instead of generating a new one.

9. Provide fit and visualization safety context

For avatar previews, the stylist can include:

- selected avatar base: male, female, or not specified
- fit status
- fit confidence
- fit warnings
- accuracy level
- grounding status
- missing visual item IDs
- visualization warnings
- footwear included flag
- accessory included or simplified warning, when applicable

All visual responses include the disclaimer:

This is a preview, not a perfect fitting.

10. Save and reuse stylist outfit recommendations

Stylist-generated outfits are persisted as `OutfitRecommendation` records with:

- source: `stylist_chat`
- request text
- item IDs
- occasion
- reuse key
- reasoning metadata

The reuse key prevents duplicate saved recommendations for the same user, item set, and occasion.

11. Let users save looks and open full previews

The current UI supports:

- saving a stylist look
- opening the full outfit preview page
- trying another look
- regenerating an avatar preview
- improving avatar size details
- asking for a simpler, more polished, more formal, or more casual version

Secondary Stylist APIs

POST /api/stylist

Legacy authenticated stylist endpoint.

Capabilities:

- accepts message
- accepts optional wardrobeSummary
- returns askStylist() output

Limitations:

- no automatic wardrobe loading
- no visualization orchestration
- no deterministic recommendation persistence

POST /api/stylist/vision

Authenticated image-question endpoint.

Input:

- imageUrl
- optional question

Capabilities:

- uses OpenAI vision model configured as wardrobeVision
- answers questions about a clothing image
- sanitizes user question

Requirements:

- OPENAI_API_KEY

Limitations:

- image URL must be supplied
- currently returns text only
- does not persist wardrobe data by itself

Security And Safety

Authentication:

- All stylist endpoints require `requireUser()`.
- Unauthenticated users receive an auth error.

Rate limits:

- `/api/stylist/chat`: 30 requests per minute per IP
- `/api/stylist`: 20 requests per minute per IP
- `/api/stylist/vision`: 10 requests per minute per IP
- stylist visualization: 6 requests per minute per user

Prompt safety:

- user text is sanitized
- recent messages are sanitized
- prompt-injection patterns are detected
- wardrobe notes, OCR, labels, and user text are treated as untrusted
- AI output is schema-validated before use
- unowned item references are stripped
- raw logs and secrets are not exposed

Fallback behavior:

- If OPENAI_API_KEY is missing, stylist chat falls back to deterministic wardrobe output.
- If AI parsing or validation fails, the system returns a safe fallback response.
- If visualization fails, chat still returns the text/outfit response where possible.

Environment Requirements

Core stylist chat:

- OPENAI_API_KEY
- configured stylist chat model via `lib/ai/models/registry.ts`
- MongoDB connection
- session/JWT configuration

Avatar and premium previews:

- OPENAI_API_KEY
- S3 storage configuration
- generated-image storage working
- avatar profile saved with consent for avatar previews
- background worker recommended for production preview jobs

Worker:

- `ENABLE_BACKGROUND_JOBS=true` for queued preview generation
- `fitpick-worker` process running in PM2

Known Limitations

- The stylist only works for authenticated users.
- Quality depends heavily on wardrobe item metadata and uploaded photos.
- It should not claim a perfect fit.
- Prompt-only fallback previews cannot guarantee exact garment appearance.
- Exact-clothing try-on requires a provider that accepts uploaded garment images as image inputs.
- Shopping advice is intentionally off by default.
- If a user has too few wardrobe items, it may return missing-category guidance instead of a full outfit.
- Avatar previews require avatar settings and consent.
- Preview generation depends on OpenAI, storage, and worker health.

QA Checklist

Test on production after deployment:

- Sign in and open /stylist.
- Ask: What should I wear today?
- Ask: Style me for church this Sunday.
- Ask: Make this more formal.
- Ask with visualization enabled and confirm a queued/generating/ready preview state.
- Confirm generated preview can be opened from the full outfit page.
- Upload shoes, bags, and accessories; confirm each category can be saved and appears in wardrobe.
- Ask for an outfit that uses shoes and accessories; confirm they appear in the item strip and saved recommendation.
- Confirm preview requests do not silently drop shoes, bags, or accessories.
- Save a stylist look.
- Try another look.
- Confirm avatar preview failure is graceful if avatar profile consent is missing.
- Confirm unowned item IDs are not returned in stylist output.
- Confirm rate limit messaging appears after repeated rapid requests.
- Confirm /api/stylist/vision returns setup error when OpenAI is missing and text analysis when configured.
- Confirm worker logs process avatar_preview_generation or outfit_preview_generation jobs when background jobs are enabled.

Production Deployment Check

After deploying the latest code:

```
npm run build:ec2
pm2 startOrRestart ecosystem.config.js --update-env
pm2 status
pm2 logs fitpick --lines 50
pm2 logs fitpick-worker --lines 50
pm2 save
```

Expected:

- fitpick online
- fitpick-worker online if ENABLE_BACKGROUND_JOBS=true
- /stylist loads for signed-in users
- /api/stylist/chat returns wardrobe-grounded responses
- avatar preview jobs queue or complete without fatal worker errors